

join_view should join all views of ranges

Document #: P2328R0
Date: 2021-03-15
Project: Programming Language C++
Audience: LEWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Abstract](#)

[2 Motivation](#)

[3 Design](#)

[3.1 But views require \$O\(1\)\$ operations?](#)

[3.2 Why does *non-propagating-cache* clear itself when it is moved from?](#)

[3.3 Is this a breaking change?](#)

[4 Wording](#)

[5 References](#)

§ 1 Abstract

This paper proposes relaxing the constraint on `join_view` to support joining ranges of prvalue non-view ranges.

§ 2 Motivation

Currently, `join_view` supports joining

- a range of glvalue ranges (whether or not the inner ranges are views), or
- a range of prvalue views.

Notably missing from the list is support for joining prvalue non-view ranges. As noted in [P2214R0] § 3.4.1, this is a fairly common use case. P2214R0 proposes to solve this problem by introducing a `views::cache_latest` adaptor, based on `views::cache1` from `range-v3`, that would cache the result of dereferencing the source iterator and produce it as a glvalue, so that the user can write `r | transform(function_returning_vector) | cache_latest | join`; it also proposes a new adaptor, commonly called `flat_map` in other languages, that would automatically add `cache_latest` to the pipeline when needed.

This solution has the appeal of generality but runs into one problem with `cache_latest`: its iterator's `operator*() const` - a const-qualified member function as required by `indirectly_readable` - can

mutate the cache, and so concurrent calls to it from different threads can result in a data race. This violates 16.4.6.10 [\[res.on.data.races\]](#). Given that `cache_latest` is always an input-only range, and can be given move-only iterators, this might not be a major problem; or perhaps `operator++` can be made to update the cache instead, with the cost of potentially unnecessary cache updates as the range is traversed if the user doesn't need to access every element. This open design issue calls into question the suitability of `cache_latest` for standardization during the C++23 time frame.

Moreover, `cache_latest` is not exactly easy to understand or discover - there's no hint that you need to use it; you just have to know of its existence, figure out why your `join` doesn't compile, and then recognize that `cache_latest` can solve the problem. The `join_view` restriction has also been a recurring source of confusion and frustration (if `#ranges` on the `cpplang` slack is any indication). By making it Just Work, we also obviate the need for a separate commonly-known-as-`flat_map` adaptor. `cache_latest` certainly has other uses beyond `join`, but those are more in the "improved performance" category than the "new functionality" category, and falls outside the Tier 1 criteria as described in P2214R0. It may be proposed for a later standard.

§ 3 Design

When dealing with a range of prvalue views, `join_view` produces input iterators, and stores a copy of the view it is currently iterating over inside the `join_view`. This paper proposes that we make it do basically the same thing with ranges of prvalue ranges; the only difference is that instead of holding the view directly, it holds the range inside a *non-propagating-cache* (which is the kebab-cased name of the class template `range-v3` uses to implement `cache1`).

A *non-propagating-cache*`<T>` is like an `optional<T>`, except that it doesn't propagate: attempts to copy or move it produces an empty cache instead. This ensures that `join_view`'s copy or move doesn't depend on what the underlying view produces. The wording below is crafted to guarantee that any prvalue range produced by `operator*` is constructed directly into the cache and will not be copied or moved.

§ 3.1 But views require $O(1)$ operations?

At any time prior to the call to `begin`, a `join_view` can be still be copied (if the underlying view can) and moved, and operations are in constant time because the cache is empty.

Since `join_view` in this case is an input range, `begin` can only be called once. After `begin` is called, the `join_view` object can no longer be used as a range, let alone a view, so the fact that destroying or moving from it may require destruction of the cached elements is irrelevant.

§ 3.2 Why does *non-propagating-cache* clear itself when it is moved from?

As a general matter, the use case for *non-propagating-cache* is to cache something that came from another thing `X` where `X` and the cache are typically data members side-by-side. Moving from `X` usually means that the contents of the cache is no longer meaningful and should be invalidated. (In particular, the various views that need to cache `begin()` in `range-v3` use *non-propagating-cache* for this purpose.)

In `join_view`'s case, it's also a serious error to move from something that is being iterated over, so clearing the cache may help catching such errors early.

§ 3.3 Is this a breaking change?

This proposal does not have to be a breaking change if we retain the existing behavior for ranges of prvalue views. However, the wording below applies the *non-propagating-cache* change unconditionally, so it changes observable behavior relative to C++20 as published in two ways:

1. `join_view` no longer default constructs the cached view when constructed;
2. `join_view` iterator increments no longer assigns to the inner view but destroys any previous view and emplaces in place instead.

Both changes appear to be desirable. The first means that we don't pay the cost of default constructing an inner view when the resulting view will just be overwritten later. It also avoids tying `join_view`'s copyability with that of the cached view when the latter is an implementation detail.

The second change means that instead of move assignment to the existing view followed by destruction of a temporary, we destroy the existing view and constructs the return value of `operator*` directly into its place. This is potentially more efficient.

This paper recommends that the proposed changes, or at least those changes that modify the handling of ranges of prvalue views, be applied retroactively to C++20 to simplify both specification and implementation. The use of `join_view` in the wild is likely rare because it is only shipped by libstdc++ as of the writing of this paper, and only as experimental.

§ 4 Wording

1. Edit 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
[...]

namespace std::ranges {

    [...]

    // [range.join], join view
    template<input_range V>
-   requires view<V> && input_range<range_reference_t<V>> &&
+   requires view<V> && input_range<range_reference_t<V>>
-   (is_reference_v<range_reference_t<V>> ||
-   view<range_value_t<V>>))
    class join_view;

    [...]
}
```

2. Add the following subclause under 24.7 [\[range.adaptors\]](#), immediately after 24.7.3 [\[range.semi.wrap\]](#):

§ 24.7.? Non-propagating cache [range.nonprop.cache]

1 Some types in this subclause are specified in terms of an exposition-only class template *non-propagating-cache*. *non-propagating-cache*<T> behaves exactly like `optional<T>` with the following differences:

(1.1) — *non-propagating-cache*<T> constrains its type parameter T with `is_object_v<T>`.

(1.2) — The copy constructor is equivalent to:

```
constexpr non-propagating-cache(non-propagating-cache const&)
    noexcept { }
```

(1.3) — The move constructor is equivalent to:

```
constexpr non-propagating-cache(non-propagating-cache&& other)
    noexcept
{
    other.reset();
}
```

(1.4) — The copy assignment operator is equivalent to:

```
constexpr non-propagating-cache& operator=(non-propagating-cache
    const& other) noexcept
{
    if (addressof(other) != this)
        reset();
    return *this;
}
```

(1.5) — The move assignment operator is equivalent to:

```
constexpr non-propagating-cache& operator=(non-propagating-cache&&
    other) noexcept
{
    reset();
    other.reset();
    return *this;
}
```

(1.6) — *non-propagating-cache*<T> has an additional member function template specified as follows:

```
template<class I>
T& emplace-deref(const I& i);    // exposition only
```

Mandates: The declaration `T t(*i);` is well-formed for some invented variable t.

Effects: Calls `reset()`. Then initializes the contained value as if direct-non-list-initializing an object of type T with the argument *i.

Returns: A reference to the new contained value.

Remarks: If an exception is thrown during the initialization of τ , **this* does not contain a value, and the previous value (if any) has been destroyed.

- 2 [*Note 1: non-propagating-cache* enables an input view to temporarily cache values as it is iterated over. — *end note*]

3. Modify 24.7.11.2 [\[range.join.view\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V>
-   requires view<V> && input_range<range_reference_t<V>> &&
+   requires view<V> && input_range<range_reference_t<V>>
-   (is_reference_v<range_reference_t<V>> ||
-   view<range_value_t<V>>)
    class join_view : public view_interface<join_view<V>> {
    private:
        using InnerRng =
            range_reference_t<V>;
        // [range.join.iterator], class template join_view::iterator
        template<bool Const>
        struct iterator;
        // [range.join.sentinel], class template join_view::sentinel
        template<bool Const>
        struct sentinel;

        V base_ = V();
-       views::all_t<InnerRng> inner_ =
            when !is_reference_v<InnerRng>
-       views::all_t<InnerRng>();
+       non_propagating_cache<remove_cv_t<InnerRng>> inner_; // exposition
            only, present only when !is_reference_v<InnerRng>
    public:
        join_view() = default;
        constexpr explicit join_view(V base);

        constexpr V base() const& requires copy_constructible<V> { return
            base_; }
        constexpr V base() && { return std::move(base_); }

        [...]
    };

    [...]
}
```

4. Modify 24.7.11.3 [\[range.join.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V>
-   requires view<V> && input_range<range_reference_t<V>> &&
+   requires view<V> && input_range<range_reference_t<V>>
-   (is_reference_v<range_reference_t<V>> ||
-   view<range_value_t<V>>)
    template<bool Const>
```

```

struct join_view<V>::iterator {
    [...]
};
}

[...]

constexpr void satisfy();           // exposition only

5   Effects: Equivalent to:

auto update_inner = [this](range_reference_t<Base> x) -> auto& {
    if constexpr (ref-is-glvalue) // x is a reference
        return x;
    else
        return (parent_ -> inner_ = views::all(std::move(x)));
};

auto update_inner = [this](const iterator_t<Base>& x) -> auto&& {
    if constexpr (ref-is-glvalue) { // *x is a reference
        return *x;
    }
    else
        return parent_ -> inner_.emplace-deref(x);
};

for (; outer_ != ranges::end(parent_ -> base_); ++outer_) {
    auto&& inner = update_inner(*outer_);
    inner_ = ranges::begin(inner);
    if (inner_ != ranges::end(inner))
        return;
}
if constexpr (ref-is-glvalue)
    inner_ = iterator_t<range_reference_t<Base>>();

[...]

constexpr iterator& operator++();

9   Let inner-range be:

(9.1) — If ref-is-glvalue is true, *outer_.
(9.2) — Otherwise, *parent_ -> inner_.

10  Effects: Equivalent to:

auto&& inner_rng = inner-range;
if (++inner_ == ranges::end(inner_rng)) {
    ++outer_;
    satisfy();
}
return *this;

```

5. Modify 24.7.11.4 [\[range.join.sentinel\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V>
-   requires view<V> && input_range<range_reference_t<V>> &&
+   requires view<V> && input_range<range_reference_t<V>>
-   (is_reference_v<range_reference_t<V>> ||
-   view<range_value_t<V>>)
    template<bool Const>
    struct join_view<V>::sentinel {
        [...]
    };
}

```

§ 5 References

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
<https://wg21.link/p2214r0>